

Some exercises on properties of predicate transformers

Consider the following simplified programming language with only assignments and sequential composition. Programs of this language only have access to two registers/variables: p and q , both stores boolean values.

$$\begin{aligned}
 Stmt & ::= variable := Expr && , \text{assignment} \\
 & \quad | Stmt ; Stmt && , \text{sequential composition} \\
 Expr & ::= variable \\
 & \quad | constant && , \text{"true" or "false"} \\
 & \quad | \neg Expr \\
 & \quad | Expr \wedge Expr \\
 variable & ::= "p" \mid "q"
 \end{aligned}$$

Consider the semantics of statements and Hoare triple shown below —these are similar to the semantics discussed in the course slides. To keep the notation simpler, we will just write, for example, $\llbracket expr \rrbracket$ and $\llbracket stmt \rrbracket$ instead of the full out $\mathcal{S}\llbracket stmt \rrbracket$ and $\mathcal{E}\llbracket expr \rrbracket$.

We will define the semantic of a statement as a function of type $state \rightarrow state$; the semantic of an expression is a function of type $state \rightarrow value$, and the semantic of a predicate (boolean expression) as a function of type $state \rightarrow \text{bool}$.

If F is a function, applying it on an argument x is denoted as $F\ x$ (a la Haskell), or $F(x)$, if that would make it clearer.

Because we only have two variables, p and q , a state s will be represented as a pair $s = \langle p : value_1, q : value_2 \rangle$. We use the notation $s.x$ and $s.y$ to get the value of the corresponding element.

The semantics of expressions:

$$\begin{aligned}
 \llbracket v \rrbracket & = (\lambda s \bullet s.v) && , \text{where } v \text{ is a variable} \\
 \llbracket c \rrbracket & = (\lambda s \bullet c) && , \text{where } c \text{ is a constant} \\
 \llbracket \neg expr \rrbracket & = (\lambda s \bullet \neg(\llbracket expr \rrbracket s)) \\
 \llbracket \neg expr_1 \wedge expr_2 \rrbracket & = (\lambda s \bullet \llbracket expr_1 \rrbracket s \wedge \llbracket expr_2 \rrbracket s)
 \end{aligned}$$

The semantics of statements and Hoare triple:

$$\begin{aligned}
 \mathcal{S}\llbracket "p" := expr \rrbracket & = (\lambda s \bullet \langle p: value, q:s.q \rangle) && , \text{where } value = \llbracket expr \rrbracket s \\
 \mathcal{S}\llbracket "q" := expr \rrbracket & = (\lambda s \bullet \langle p:s.p, q: value \rangle) && , \text{where } value = \llbracket expr \rrbracket s \\
 \mathcal{S}\llbracket S_1; S_2 \rrbracket & = (\lambda s \bullet \llbracket S_2 \rrbracket (\llbracket S_1 \rrbracket s)) \\
 \llbracket \{P\} stmt \{Q\} \rrbracket & = (\forall s :: \llbracket P \rrbracket s \Rightarrow \llbracket Q \rrbracket (\llbracket stmt \rrbracket s))
 \end{aligned}$$

Definition of wlp is as usual:

$$\begin{aligned}
 wlp\ "p" := expr\ Q & = Q[expr/p] \\
 wlp\ "q" := expr\ Q & = Q[expr/q] \\
 wlp\ (stmt_1; stmt_2)\ Q & = wlp\ stmt_1\ (wlp\ stmt_2\ Q)
 \end{aligned}$$

Substitution e/v is also defined as usual, but then limited to the syntax of expression of the above programming language.

$$\begin{array}{ll}
c[e/v] & = c \\
\text{"p"}[e/p] & = e \\
\text{"p"}[e/q] & = p \\
\text{"q"}[e/p] & = q \\
\text{"q"}[e/q] & = e \\
(\neg expr)[e/v] & = \neg(expr[e/v]) \\
(expr_1 \wedge expr_2)[e/v] & = expr_1[e/v] \wedge expr_2[e/v]
\end{array}
\quad , v \text{ is either "p" or "q"}$$

1. **wlp** is **sound and complete** if for any post-condition Q , for any statement $stmt$, and for any state s , we have:

s satisfies **wlp** $stmt$ Q if and only if the state $t = \llbracket stmt \rrbracket s$ satisfies the post-condition Q .

This can be written more formally as the following equivalence:

$$\llbracket \text{wlp } stmt \ Q \rrbracket s \equiv \llbracket Q \rrbracket (\llbracket stmt \rrbracket s)$$

Prove that the above **wlp** is sound and complete over assignments.

2. Assuming that **wlp** is sound and complete over $stmt_1$ and $stmt_2$, prove that it is also sound and complete over $stmt_1; stmt_2$.
3. Prove that **wlp** is sound and complete over any statement in the above given programming language.
4. Prove that **wlp** is monotonic. That is if $P \Rightarrow Q$ is valid, then the implication $\text{wlp } stmt \ P \Rightarrow \text{wlp } stmt \ Q$ is also valid.
5. Prove that **wlp** is distributive over $\wedge$. So, prove that this hold for any statement $stmt$ and post-conditions P and Q :

$$\text{wlp } stmt \ (P \wedge Q) = \text{wlp } stmt \ P \wedge \text{wlp } stmt \ Q$$